

TECHNICKÁ UNIVERZITA V KOŠICIACH
Fakulta elektrotechniky a informatiky

**Porovnanie transformerových architektúr pri detekcii
nenávisťného prejavu na sociálnych sieťach**
Bakalárska práca

Študijný program: Hospodárska informatika
Študijný odbor: Informatika
Školiace pracovisko: Ústav umelej inteligencie
Školiteľ: prof. Ing. Kristína Machová, PhD.
Konzultant: Ing. Peter Demeter

Obsah

Zoznam obrázkov	3
1. Popis programu	4
1.1. Import potrebných knižníc	4
1.2. Načítanie a predspracovanie dát	5
1.3. Trieda SimpleDataset	6
1.4. Model T5Classifier	7
1.5. Model DebertaClassifier	7
1.6. Funkcia save_fold_loss_curve	8
1.7. Funkcia train_model_advanced	8
1.8. Krížová validácia a výber najlepšieho modelu	9
1.9. Funkcia get_detailed_report	10

Zoznam obrázkov

Obr. 1 Import potrebných knižníc pre BERT model	4
Obr. 2 Import potrebných knižníc pre T5 model	5
Obr. 3 Predspracovanie dát	5
Obr. 4 Ukážka kódu - Trieda SimpleDataset	6
Obr. 5 Ukážka kódu - Model T5Classifier	7
Obr. 6 Ukážka kódu - Model DebertaClassifier	7
Obr. 7 Ukážka kódu - Funkcia save_fold_loss_curve	8
Obr. 8 Ukážka kódu - Trénovanie modelu DeBERTa – 1 časť	8
Obr. 9 Ukážka kódu - Trénovanie modelu DeBERTa – 2 časť	9
Obr. 10 Ukážka kódu - Krížová validácia DeBERTa – 1 časť	9
Obr. 11 Ukážka kódu - Krížová validácia DeBERTa – 2 časť	10
Obr. 12 Ukážka kódu - Funkcia get_detailed_report	10

1. Popis programu

Programová časť bakalárskej práce sa zameriava na porovnanie transformerových architektúr pri detekcii nenávistného prejavu na sociálnych sieťach. Implementácia využíva dátový súbor labeled_data.csv, ktorý obsahuje textové príspevky zo sociálnej siete a cieľový atribút class. Triedy sú definované ako Hate, Offensive a Neither. V praktickej časti boli implementované a tréňované najmä modely T5-small a DeBERTa-v3-base. Použitá je päťnásobná stratifikovaná krížová validácia, ukladanie najlepších modelov a vyhodnotenie pomocou klasifikačných metrík. Riešenie je implementované v jazyku Python s využitím knižníc PyTorch, Transformers, scikit-learn, pandas, NumPy a Matplotlib. Program je rozdelený do logických častí: import knižníc, načítanie a čistenie dát, definícia datasetovej triedy, definícia modelov, tréňovanie s mechanizmom early stopping, krížová validácia a finálne vyhodnotenie. Nasledujúce podkapitoly opisujú jednotlivé časti programu a obsahujú ukážky zdrojového kódu.

1.1. Import potrebných knižníc

Na začiatku sa importujú knižnice potrebné na prácu s dátami, vytváranie neurónových sietí, tokenizáciu textu, tréňovanie transformerových modelov a vyhodnocovanie výsledkov. Pre model T5-small sa používajú T5Tokenizer a T5EncoderModel, zatiaľ čo implementácia DeBERTa-v3 využíva univerzálne triedy AutoTokenizer a AutoModel z knižnice Transformers. Súčasne sa inicializuje výpočtové zariadenie a program automaticky využíva GPU, ak je k dispozícii.

```
!pip install transformers datasets scikit-learn tqdm matplotlib seaborn torch

import os
import re
import torch
import torch.nn as nn
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from torch.utils.data import Dataset, DataLoader
from transformers import AutoTokenizer, BertModel
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import classification_report, confusion_matrix
from tqdm.auto import tqdm

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

if not os.path.exists('bert_checkpoints/plots'):
    os.makedirs('bert_checkpoints/plots')

print(f"Zariadenie: {device}")
```

Obr. 1 Import potrebných knižníc pre BERT model

```

!pip install transformers datasets scikit-learn joblib tqdm matplotlib seaborn

import os
import re
import torch
import torch.nn as nn
import pandas as pd
import numpy as np
import joblib
import matplotlib.pyplot as plt
import seaborn as sns
from torch.utils.data import Dataset, DataLoader
from transformers import T5Tokenizer, T5EncoderModel
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.ensemble import GradientBoostingClassifier
from tqdm.auto import tqdm

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

```

Obr. 2 Import potrebných knižníc pre T5 model

1.2. Načítanie a predspracovanie dát

Dáta sa načítajú zo súboru labeled_data.csv. Pred začiatkom tréningu sa text každého tweetu normalizuje: premení sa na malé písmená, odstránia sa mená používateľov, URL adresy, tagy retweetu, HTML znaky a nadbytočné medzery. Po vyčistení sa odstránia duplikáty a prázdne záznamy. Výsledkom tejto fázy sú vstupné texty X a cieľové triedy y, ktoré sa následne používajú v ďalších krokoch spracovania. Vokrížovej validácii.

```

df = pd.read_csv('labeled_data.csv')

def clean_text(text):
    text = text.lower()
    text = re.sub(r'@[w-]+', '', text)
    text = re.sub(r'https?://[S+|www|.S+]', '', text)
    text = re.sub(r'\brt\b', '', text)
    text = re.sub(r'&|&#\d+', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

df['clean_tweet'] = df['tweet'].apply(clean_text)
df = df.drop_duplicates(subset=['clean_tweet'])
df = df[df['clean_tweet'] != ''].dropna()
X = df['clean_tweet']
y = df['class']

print(f"Celkový počet záznamov po čistení: {len(df)}")

```

Obr. 3 Predspracovanie dát

1.3. Trieda SimpleDataset

Trieda SimpleDataset zabezpečuje prevod textov do formátu vhodného pre PyTorch DataLoader. Každý záznam je tokenizovaný pomocou príslušného tokenizátora modelu s využitím vyplňania, skrátenia a maximálnej dĺžky sekvencie 64 tokenov. Metóda `__getitem__` vracia slovník obsahujúci `input_ids`, `attention_mask` a `label`. Tento jednotný formát umožňuje použitie rovnakej triedy dátového súboru pre modely T5-small aj DeBERTa-v3.

```
class SimpleDataset(Dataset):

    def __init__(self, texts, labels, tokenizer):
        self.texts = texts.values
        self.labels = labels.values
        self.tokenizer = tokenizer

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        text = str(self.texts[idx])
        encoding = self.tokenizer(
            text,
            padding='max_length',
            truncation=True,
            max_length=64,
            return_tensors='pt'
        )

        return {
            'ids': encoding['input_ids'].flatten(),
            'mask': encoding['attention_mask'].flatten(),
            'label': torch.tensor(self.labels[idx], dtype=torch.long)
        }
```

Obr. 4 Ukážka kódu - Trieda SimpleDataset

1.4. Model T5Classifier

Model T5Classifier využíva kódovač z vopred vytrénovaného modelu t5-small. Úloha detekcie nenávistných prejavov je v tejto implementácii formulovaná ako klasifikačná úloha. Z výstupu kódovača sa vypočíta priemerná reprezentácia tokenov, ktorá sa následne odošle do lineárnej klasifikačnej vrstvy predpovedajúcej jednu z troch tried.

```
class T5Classifier(nn.Module):
    def __init__(self, num_classes=3):
        super(T5Classifier, self).__init__()
        self.encoder = T5EncoderModel.from_pretrained("t5-small")
        self.classifier = nn.Linear(self.encoder.config.d_model, num_classes)

    def forward(self, ids, mask):
        outputs = self.encoder(input_ids=ids, attention_mask=mask)
        mean_output = torch.mean(outputs.last_hidden_state, dim=1)
        logits = self.classifier(mean_output)
        return logits
```

Obr. 5 Ukážka kódu - Model T5Classifier

1.5. Model DebertaClassifier

Model DebertaClassifier využíva vopred vytrénovaný model microsoft/deberta-v3-base. Výstup poslednej skrytej vrstvy je spracovaný tak, že sa používa reprezentácia prvého tokenu ako zovšeobecnená reprezentácia celej sekvencie. Pred klasifikáciou sa aplikuje dropout s hodnotou 0,3 na zníženie rizika pretrénovania. Výsledná lineárna vrstva mapuje skrytú reprezentáciu na tri cieľové triedy.

```
class DebertaClassifier(nn.Module):
    def __init__(self, num_classes=3):
        super(DebertaClassifier, self).__init__()
        self.deberta = AutoModel.from_pretrained(model_name)
        self.deberta = self.deberta.float()
        self.dropout = nn.Dropout(0.3)
        self.classifier = nn.Linear(self.deberta.config.hidden_size, num_classes)
        self.classifier = self.classifier.float()

    def forward(self, ids, mask):
        outputs = self.deberta(input_ids=ids, attention_mask=mask)
        last_hidden_state = outputs.last_hidden_state
        cls_output = last_hidden_state[:, 0, :]
        out = self.dropout(cls_output)
        return self.classifier(out)
```

Obr. 6 Ukážka kódu - Model DebertaClassifier

1.6. Funkcia save_fold_loss_curve

Počas tréovania sa pre každý fold ukladajú hodnoty trénovacej a validačnej chyby. Funkcia save_fold_loss_curve z týchto hodnôt vytvára graf priebehu kriviek loss. Tento graf sa ukladá do adresára s checkpointmi modelu, čo umožňuje spätnú analýzu stability tréovania a prípadného pretréovania.

```
def save_fold_loss_curve(history, fold_num):
    plt.figure(figsize=(4, 2.5))
    plt.plot(history['train'], label='Train Loss', linewidth=1.5, marker='o', markersize=4)
    plt.plot(history['val'], label='Val Loss', linewidth=1.5, marker='s', markersize=4)
    plt.title(f'DeBERTa Loss: Fold {fold_num}', fontsize=9)
    plt.xlabel('Epoch', fontsize=8)
    plt.ylabel('Loss', fontsize=8)
    plt.xticks(range(len(history['train'])), range(1, len(history['train']) + 1), fontsize=7)
    plt.legend(fontsize=7)
    plt.grid(True, linestyle='--', alpha=0.5)
    plot_path = f'deberta_checkpoints/plots/fold_{fold_num}_loss_curves.png'
    plt.savefig(plot_path, dpi=300, bbox_inches='tight')
    plt.close()
```

Obr. 7 Ukážka kódu - Funkcia save_fold_loss_curve

1.7. Funkcia train_model_advanced

Funkcia train_model_advanced predstavuje hlavný cyklus tréovania. V každej epoche sa model prepne do režimu tréovania, vypočíta sa hodnota funkcie strát CrossEntropyLoss a parametre modelu sa aktualizujú pomocou optimalizátora AdamW. Po tréovaní nasleduje fáza validácie bez výpočtu gradientov. Ak sa validačná chyba zlepšuje, uloží sa aktuálny stav modelu. Ak sa chyba nezlepšuje počas stanoveného počtu epoch, aktivuje sa mechanizmus early stopping.

```
def train_model_advanced(model, train_loader, val_loader, fold, epochs=5, patience=2):

    optimizer = torch.optim.AdamW(model.parameters(), lr=1e-5)
    criterion = torch.nn.CrossEntropyLoss()
    best_val_loss = float('inf')
    epochs_no_improve = 0
    history = {'train': [], 'val': []}

    print(f"Tréovanie DeBERTa Fold {fold}")

    for epoch in range(epochs):
        model.train()
        total_train_loss = 0
        loop = tqdm(train_loader, desc=f"Fold {fold} | Ep {epoch+1}")

        for batch in loop:
            optimizer.zero_grad()
            ids, mask = batch['ids'].to(device), batch['mask'].to(device)
            labels = batch['label'].to(device)
            outputs = model(ids, mask)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            total_train_loss += loss.item()
```

Obr. 8 Ukážka kódu - Tréovanie modelu DeBERTa – 1 časť

```

model.eval()
total_val_loss, correct = 0, 0
with torch.no_grad():
    for batch in val_loader:
        ids, mask = batch['ids'].to(device), batch['mask'].to(device)
        labels = batch['label'].to(device)
        outputs = model(ids, mask)
        total_val_loss += criterion(outputs, labels).item()
        correct += (torch.argmax(outputs, dim=1) == labels).sum().item()

avg_train = total_train_loss / len(train_loader)
avg_val = total_val_loss / len(val_loader)
history['train'].append(avg_train)
history['val'].append(avg_val)

print(f"Ep {epoch+1}: Train {avg_train:.4f} | Val {avg_val:.4f} | Acc {correct/len(val_loader.dataset):.4f}")

if avg_val < best_val_loss:
    best_val_loss = avg_val
    torch.save(model.state_dict(), f"deberta_checkpoints/best_model_fold_{fold}.pt")
    epochs_no_improve = 0
else:
    epochs_no_improve += 1

if epochs_no_improve >= patience:
    print("Early Stopping!")
    break
return history

```

Obr. 9 Ukážka kódu - Trénovanie modelu DeBERTa – 2 časť

1.8. Krížová validácia a výber najlepšieho modelu

Na objektívne hodnotenie sa používa StratifiedKFold s piatimi zložkami, náhodným usporiadaním a pevnou hodnotou random_state=42. Stratifikácia zachováva pomer tried v jednotlivých častiach dát. Pre každý fold sa vytvorí samostatný DataLoader na tréovanie a validáciu, inicializuje sa nový model a spustí sa tréovanie. Po dokončení všetkých foldov sa porovnajú validačné chyby a najlepší model sa uloží ako hlavný checkpoint.

```

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
tokenizer = AutoTokenizer.from_pretrained(model_name)
global_best_loss = float('inf')
best_val_indices = None

for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):
    print(f"\n===== DEBERTA FOLD {fold+1} =====")
    train_ds = SimpleDataset(X.iloc[train_idx], y.iloc[train_idx], tokenizer)
    val_ds = SimpleDataset(X.iloc[val_idx], y.iloc[val_idx], tokenizer)
    train_loader = DataLoader(train_ds, batch_size=16, shuffle=True)
    val_loader = DataLoader(val_ds, batch_size=16)
    model = DebertaClassifier().to(device).float()
    history = train_model_advanced(model, train_loader, val_loader, fold=fold+1)
    save_fold_loss_curve(history, fold_num=fold+1)
    path_to_current_best = f"deberta_checkpoints/best_model_fold_{fold+1}.pt"
    model.load_state_dict(torch.load(path_to_current_best))

```

Obr. 10 Ukážka kódu - Krížová validácia DeBERTa – 1 časť

```

current_fold_loss = 0
with torch.no_grad():
    for batch in val_loader:
        logits = model(batch['ids'].to(device), batch['mask'].to(device))
        loss = torch.nn.CrossEntropyLoss()(logits, batch['label'].to(device))
        current_fold_loss += loss.item()

avg_current_fold_loss = current_fold_loss / len(val_loader)
print(f"Finaálna strata pre Fold {fold+1}: {avg_current_fold_loss:.4f}")

if avg_current_fold_loss < global_best_loss:
    global_best_loss = avg_current_fold_loss
    najlepsi_fold_cislo = fold + 1
    best_val_indices = val_idx
    torch.save(model.state_dict(), "deberta_checkpoints/absolute_best_deberta_model.pt")
    print(f"--> Nový absolútne najlepší model nájdený vo Folde {fold+1}!")

```

Obr. 11 Ukážka kódu - Krížová validácia DeBERTa – 2 časť

1.9. Funkcia get_detailed_report

Funkcia `get_detailed_report` slúži na konečné vyhodnotenie modelu. Model sa prepne do režimu vyhodnocovania, pre validačné dáta sa vypočítajú predpovede a následne sa pomocou `classification_report` vygenerujú metriky pre triedy Hate, Offensive a Neither. Sledované metriky zahŕňajú presnosť (precision), recall, F1-skóre a celkovú presnosť.

```

def get_detailed_report(model, loader, title="VÝSLEDOK"):
    model.eval()
    all_preds, all_labels = [], []
    with torch.no_grad():
        for batch in loader:
            logits = model(batch['ids'].to(device), batch['mask'].to(device))
            all_preds.extend(torch.argmax(logits, dim=1).cpu().numpy())
            all_labels.extend(batch['label'].numpy())

    print(f"\n=== {title} ===")
    print(classification_report(all_labels, all_preds, target_names=['Hate', 'Offensive', 'Neither'], digits=4))
    return all_labels, all_preds

final_model = DebertaClassifier().to(device)
final_model.load_state_dict(torch.load("deberta_checkpoints/absolute_best_deberta_model.pt"))
test_loader = DataLoader(SimpleDataset(X.iloc[best_val_indices], y.iloc[best_val_indices], tokenizer), batch_size=16)
y_true, y_pred = get_detailed_report(final_model, test_loader, "FINÁLNY REPORT: DeBERTa-v3")

```

Obr. 12 Ukážka kódu - Funkcia get_detailed_report